

Aerotwin DRL Resource Allocation System — Technical Specification v2.0

Document Purpose: Transition from single-flight config generator to schedule-aware capacity planner.

Date: 2026-06-05

Status: Production model v1 locked. Retrain required for v2.

Target: IATA compliance optimization across multi-flight schedules (1–5 flights, 50–500 PAX/flight).

Table of Contents

1. Executive Summary
2. Current System (v1 — Locked)
3. Target System (v2 — Required Changes)
4. Simulation Engine Specification
5. File Inventory & Responsibilities
6. Unity vs Python DES Validation Checklist
7. Training Specification
8. Inference & API Specification
9. Known Issues & Fixes
10. Appendix: Reward Math

1. Executive Summary

What We Have (v1)

A PPO policy trained on **single-flight scenarios** (~200 PAX, 1 ABS profile per episode). It outputs a 130-dimensional action vector decoded into an AERO config. Best achieved: **-180 raw reward** (vs IQL baseline -497, vs random -36,000). The model **cannot handle variable PAX or multi-flight schedules** because the observation space does not contain scale information.

What We Need (v2)

A **schedule-aware capacity planner** that:

- Accepts a flight schedule (1–5 flights, variable PAX 50–500 per flight)
- Observes total demand, peak hour, flow mix, and ABS profiles
- Outputs **one AERO config** optimized for the entire schedule
- Maximizes **per-checkpoint IATA compliance rate** (not scalar reward)
- Scales staffing and capacity with total PAX

Why v1 Fails for Multi-Flight

The current observation is **7-dimensional** (ABS passenger class proportions only). The policy has no concept of:

- Total PAX count
- Number of simultaneous flights
- Peak hour density
- Arrival vs departure ratio

When given a 2,000-PAX rush hour, it outputs the same staffing as for 200 PAX. The simulation collapses.

2. Current System (v1 — Locked)

2.1 Architecture Overview

ABS Config (1 flight) → PPO Policy (130-dim) → Python DES Sim (200 PAX)

2.2 Observation Space (v1)

```
shape=(7,) # From parse_iql_v2.featurize_absconfig()
# 0-6: Normalized passenger class proportions from ABS weights
```

2.3 Action Space (v1)

```
shape=(130,) # 13 checkpoints × 10 features per checkpoint
# Features per checkpoint (indices relative to ck_offset):
# 0: num_stations
# 1: total_staffing
# 2: total_queue_cap
# 3: mean_efficiency
# 4: mean_service_time
# 5: has_buffer
# 6: has_parallel
# 7: has_split
# 8: total_tasks
# 9: mean_cond_prob
```

2.4 Reward Function (v1 — BROKEN, DO NOT REUSE)

```
# Log-normalized reward (destroys per-checkpoint shaping)
def normalize_reward(raw_reward, scaler):
    shift = scaler["shift"] # ~+1000
    log_mean = scaler["log_mean"]
    log_std = scaler["log_std"]
    val = max(float(raw_reward) + shift, 1e-6)
    r_log = np.log(val)
    return float((r_log - log_mean) / log_std)

# Per-checkpoint shaping (+12 green, -20 red) is INVISIBLE after log compression
# explained_variance stays at ~0.00 because value net sees noise
```

2.5 Episode Structure (v1)

```
# Single-step episode
obs = featurize_absconfig(abs_path)      # 7-dim
action = policy.predict(obs)             # 130-dim
aero = action_to_aeroconfig_structural(...) # Decode
stats = run_sim_in_memory(aero, abs_path, seed, pax_count=50) # 1 sim
reward = normalize_reward(raw, scaler)    # Log-scaled
return next_obs, reward, True, False, {} # done=True immediately
```

2.6 Decoder (v1 — GRADIENT KILLER)

```
# In action_to_aeroconfig_structural / action_to_aeroconfig_dict:
t_staff = max(n, int(round(feats[FEAT["total_staffing"]])))
t_cap = max(n * 5, int(round(feats[FEAT["total_queue_cap"]])))
# int(round()) creates zero gradient — PPO cannot learn fine adjustments
```

2.7 Training Hyperparameters (v1 — Fast Iteration)

```
PAX_COUNT = 50
TOTAL_STEPS = 5_000
n_steps = 64
batch_size = 64
n_epochs = 4
clip_range = 0.2
ent_coef = 0.0001
vf_coef = 0.5
learning_rate = get_linear_fn(1e-4, 1e-5, 1.0)
```

2.8 Warm-Start (v1)

```
# IQL behavior cloning with critical [-1,1] → [0,1] conversion
action = iql_model.predict(obs[np.newaxis, :])[0] # [-1,1] from tanh
action_01 = np.clip((action + 1.0) / 2.0, 0.0, 1.0) # CRITICAL FIX
# BC MSE converged to 0.000109 — warm-start successful
```

2.9 Production Lock Status

- **Model file:** ppo_v1_production.zip (best: -180.49)
- **IQL baseline:** model_final.d3 (warm-start artifact, -497)
- **Action scaler:** action_scaler.json (REQUIRED for inference)
- **Reward scaler:** reward_scaler.json (v1 artifact, NOT needed for v2)
- **Baseline config:** baseline.json (airport layout template)

3. Target System (v2 — Required Changes)

3.1 Architecture Overview

```
Flight Schedule (1-5 flights) + ABS profiles → PPO Policy (130-dim) → Python DES Sim (Aggregate)
```

3.2 Observation Space (v2 — NEW)

```
shape=(15,) # Expanded to include scale and schedule information

# Dimensions:
# 0-6: ABS passenger class proportions (normalized) [from featurize_absconfig]
# 7:   total_pax / 2000.0                        [total demand scale]
# 8:   num_flights / 10.0                        [schedule density]
# 9:   peak_pax / 1000.0                         [peak hour intensity]
# 10:  dep_pax_ratio                             [departure proportion]
# 11:  arr_pax_ratio                             [arrival proportion]
# 12:  first_flight_hour / 24.0                  [time of day]
# 13:  unique_abs_count / 5.0                    [ABS diversity]
# 14:  avg_service_time_factor / 100.0           [optional: task complexity]
```

3.3 Action Space (v2 — UNCHANGED)

```
shape=(130,) # Same 13 checkpoints × 10 features
# Same decoder logic, but with continuous values during training
```

3.4 Episode Structure (v2 — NEW)

```
# Multi-flight aggregate episode
schedule = schedule_sampler.sample()      # 1-5 flights
obs = schedule_to_obs(schedule)           # 15-dim
action = policy.predict(obs)              # 130-dim
aero = action_to_aeroconfig_structural(...) # Decode

# Run ALL flights through the same AERO config
all_records = []
for flight in schedule:
    records = sim.run_flight(
        aero_dict=aero,
        abs_config=flight["abs_path"],
        pax_count=flight["pax_count"],
        flow=flight["flow"],
        seed=flight["flight_id"]
    )
    all_records.extend(records)

# Aggregate stats across full schedule
agg_stats = sim.compute_aggregate_stats(all_records)

# NEW: Direct IATA compliance reward (NO log normalization)
raw_reward = compute_iata_reward(agg_stats)
norm_reward = raw_reward / 100.0          # Simple linear scaling for PPO

# Next episode: NEW random schedule
next_schedule = schedule_sampler.sample()
next_obs = schedule_to_obs(next_schedule)
```

```

return next_obs, norm_reward, True, False, {
    "raw_reward": raw_reward,
    "compliance_rate": compute_compliance_rate(agg_stats),
    "per_checkpoint": agg_stats.get("per_checkpoint", {}),
    "total_pax": total_pax
}

```

3.5 Reward Function (v2 — NEW)

CRITICAL: Remove log normalization. Remove reward_scaler dependency.

```

def compute_iata_reward(stats):
    per_ck = stats.get("per_checkpoint", {})
    if not per_ck:
        return -1000.0

    total_bonus = 0.0
    total_penalty = 0.0
    num_checked = 0

    for cid, data in per_ck.items():
        los = IATA_LOS.get(cid)
        if not los or not los.economy:
            continue

        p95 = data.get("p95", 0.0)
        lo, hi = los.economy
        num_checked += 1

        if lo <= p95 <= hi:
            total_bonus += 50.0          # Strong positive for green
        elif p95 < lo:
            total_penalty += 3.0 * (lo - p95) / lo  # Mild over-design penalty
        else:
            total_penalty += 25.0 * (p95 - hi) / hi # Strong under-design penalty

    # Throughput incentive
    throughput = stats.get("total_throughput", 0)
    throughput_bonus = throughput * 0.05

    # Completion incentive
    completion = stats.get("completion_rate", 0)
    completion_bonus = completion * 20.0

    return total_bonus - total_penalty + throughput_bonus + completion_bonus

# Normalization for PPO stability ONLY (simple linear)
def normalize_for_ppo(raw_reward):
    return raw_reward / 100.0  # Scale to roughly [-5, 5] range

```

3.6 Compliance Rate (v2 — PRIMARY METRIC)

```
def compute_compliance_rate(stats):
    per_ck = stats.get("per_checkpoint", {})
    green = 0
    total = 0
    for cid, data in per_ck.items():
        los = IATA_LOS.get(cid)
        if not los or not los.economy:
            continue
        total += 1
        lo, hi = los.economy
        if lo <= data.get("p95", 0) <= hi:
            green += 1
    return green / total if total > 0 else 0.0
```

Target: >90% compliance rate across diverse schedules.

3.7 Decoder (v2 — CONTINUOUS FOR TRAINING)

```
# During TRAINING: continuous values (no int rounding)
t_staff = max(float(n), feats[FEAT["total_staffing"]])
t_cap = max(float(n * 5), feats[FEAT["total_queue_cap"]])
eff = float(np.clip(feats[FEAT["mean_efficiency"]], 0.1, 1.0))

for i, st in enumerate(op_st):
    st["Staffing_No"] = t_staff / n + (t_staff % n if i == 0 else 0)
    st["Max_Queue_Cap"] = t_cap / n + (t_cap % n if i == 0 else 0)
    st["Efficiency_Factor"] = round(eff, 3)

# During INFERENCE: round to integers for real deployment
st["Staffing_No"] = int(round(st["Staffing_No"]))
st["Max_Queue_Cap"] = int(round(st["Max_Queue_Cap"]))
```

3.8 Schedule Sampler (v2 — NEW)

```
class ScheduleSampler:
    def __init__(self, abs_config_dir, api_schedule=None):
        self.abs_paths = sorted(Path(abs_config_dir).glob("absconfig_*.json"))
        self.api_schedule = api_schedule

    def sample(self):
        num_flights = random.randint(1, 5)
        schedule = []
        for i in range(num_flights):
            pax = random.randint(50, 500)
            flow = random.choice(["departure", "arrival"])
            hour = random.randint(6, 22)
            abs_path = random.choice(self.abs_paths)
            schedule.append({
                "flight_id": i,
```

```

        "pax_count": pax,
        "flow": flow,
        "hour": hour,
        "abs_path": abs_path,
        "abs_id": Path(abs_path).stem,
        "abs_weights": self._get_abs_weights(abs_path)
    })
    return schedule

def from_api(self, api_flights):
    return [{
        "flight_id": f["flight_id"],
        "pax_count": f["pax_count"],
        "flow": f["flow"],
        "hour": datetime.fromisoformat(f["scheduled_time"]).hour,
        "abs_path": self._resolve_abs(f["flight_type"]),
        "abs_id": f["flight_type"],
        "abs_weights": self._get_abs_weights_by_id(f["flight_type"])
    } for f in api_flights]

```

3.9 Training Hyperparameters (v2)

```

PAX_COUNT = None          # Now variable per flight in schedule
TOTAL_STEPS = 25_000      # Convergence run
EVAL_EVERY = 500

ppo = PPO(
    policy="MlpPolicy",
    env=env,
    learning_rate=get_linear_fn(3e-4, 1e-5, 1.0),
    n_steps=512,
    batch_size=128,
    n_epochs=4,
    gamma=0.99,
    clip_range=0.1,
    ent_coef=0.01,
    vf_coef=0.5,
    policy_kwargs=dict(net_arch=[256, 256, 256]),
    verbose=1,
    tensorboard_log="./ppo_airport_v2/"
)

```

3.10 Warm-Start (v2 — NO IQL)

IQL warm-start is **discarded**. IQL was trained on single-flight, fixed-PAX data and is irrelevant for schedule-aware planning.

New warm-start: Linear scaling baseline or random initialization. PPO will learn quickly with the direct reward.

4. Simulation Engine Specification

4.1 Python DES Engine (des_engine.py)

Core Functions:

```
def load_aero_config(path: str) -> dict:
    """Load AERO JSON and return {flow: checkpoints}."""

def load_abs_config(path: str) -> dict:
    """Load ABS JSON and return passenger profiles."""

def generate_passengers(profiles: dict, pax_count: int, seed: int) -> list:
    """Generate passenger list with gamma-distributed arrival times."""
    # CRITICAL: Arrival times follow gamma distribution to imitate real arrivals
    # Parameters must match Unity implementation

def simulate(checkpoints: list, passengers: list, seed: int) -> list:
    """Run DES for one flow. Returns list of passenger records."""
    # Each passenger traverses checkpoints according to flow graph
    # Queue discipline: FIFO with Max_Queue_Cap enforcement
    # Service time: Avg_Service_Time * random factor / Efficiency_Factor

def run_single(aero_path: str, abs_path: str, seed: int, pax_count: int) -> tuple:
    """Convenience wrapper for single-flight simulation."""
```

Passenger Record Fields:

```
@dataclass
class PassengerRecord:
    passenger_id: int
    checkpoint_id: str
    station_id: str
    arrival_time: float      # When passenger arrived at checkpoint
    service_start: float     # When service began
    wait_time: float         # service_start - arrival_time
    service_time: float      # How long service took
    departure_time: float    # service_start + service_time
    passenger_class: str
    flow_type: str
```

Aggregate Stats Computation:

```
def compute_aggregate_stats(all_records: list) -> dict:
    """Compute stats across all flights in schedule."""
    # Group by checkpoint
    # Compute: mean, median, p95, p99 wait times
    # Compute: throughput (passengers served)
    # Compute: completion rate (passengers who reached terminal)
    # Compute: per-station breakdown
    return {
        "per_checkpoint": {
            "1ST-SEC": {
```



```

        "mqt": mean_queue_time,
        "p95": np.percentile(wait_times, 95),
        "throughput": count,
        "breach_count": sum(1 for w in wait_times if w > threshold)
    },
    ...
},
"total_throughput": total,
"completion_rate": completed / total,
"mean_journey_min": mean_total_journey / 60.0,
"p95_journey_min": p95_total_journey / 60.0
}

```

4.2 IATA Reward Computation (iata_reward.py)

IATA Level of Service Bands:

```

@dataclass
class IATALOS:
    economy: tuple[float, float] # (min_acceptable, max_acceptable) in minutes
    business: tuple[float, float]
    first: tuple[float, float]

IATA_LOS = {
    "1ST-SEC": IATALOS(economy=(15, 30), business=(10, 20), first=(5, 15)),
    "CHK-BAG": IATALOS(economy=(10, 25), business=(8, 18), first=(5, 12)),
    "DIG-CHK": IATALOS(economy=(5, 15), business=(3, 10), first=(2, 8)),
    "SLF-BAG": IATALOS(economy=(5, 12), business=(3, 10), first=(2, 8)),
    "PAS-CHK": IATALOS(economy=(5, 15), business=(3, 10), first=(2, 8)),
    "2ND-SEC": IATALOS(economy=(10, 25), business=(8, 20), first=(5, 15)),
    "DEPARTING-TERMINAL": IATALOS(economy=(0, 60), business=(0, 45), first=(0, 30)),
    "BRD-GAT": IATALOS(economy=(5, 15), business=(3, 10), first=(2, 8)),
    "ARV-GAT": IATALOS(economy=(10, 25), business=(8, 20), first=(5, 15)),
    "ARV-TERM": IATALOS(economy=(0, 60), business=(0, 45), first=(0, 30)),
    "PAS-CTRL": IATALOS(economy=(10, 30), business=(8, 25), first=(5, 20)),
    "BAG-CLM": IATALOS(economy=(15, 45), business=(10, 35), first=(8, 25)),
    "EXIT-SEC": IATALOS(economy=(10, 30), business=(8, 25), first=(5, 20)),
}

```

Reward Computation (v2):

```

def compute_reward_iata(stats, csv_df=None):
    per_ck = stats.get("per_checkpoint", {})
    breach_total = 0.0
    breach_worst = 0.0

    for cid, data in per_ck.items():
        los = IATA_LOS.get(cid)
        if not los or not los.economy:
            continue
        p95 = data.get("p95", 0.0)
        lo, hi = los.economy

```

```

    if p95 > hi:
        breach = (p95 - hi) / hi
        breach_total += breach
        breach_worst = max(breach_worst, breach)

raw_reward = -10.0 * breach_total - 50.0 * breach_worst
completion = stats.get("completion_rate", 0)
raw_reward += 100.0 * completion

info = {
    "breach_total": breach_total,
    "breach_worst": breach_worst,
    "completion": completion
}

return raw_reward, info

```

4.3 Gamma Distribution Parameters

The arrival time distribution must match between Python and Unity:

```

# Python implementation
def gamma_arrival_time(flight_time, num_pax, shape=2.0, scale=15.0):
    """Generate arrival times relative to flight departure/arrival."""
    # Passengers arrive in a window before the flight event
    # shape=2.0, scale=15.0 (minutes) means most arrive 15-30 min before
    return np.random.gamma(shape, scale, num_pax)

```

Unity must use identical parameters. This is a critical validation point.

5. File Inventory & Responsibilities

5.1 Python Inference Service (NEW REPO: **aerotwin-inference/**)

File	Source	Purpose	v2 Changes
main.py	NEW	FastAPI app with /infer, /simulate, /health	Add schedule-aware endpoints
requirements.txt	NEW	torch, stable-baselines3, fastapi, uvicorn, gymnasium, numpy, pandas	Add d3rlpy only if keeping IQL comparison
Dockerfile	NEW	Container for Python service	Standard Python 3.11 slim
ppo_v2.zip	TRAIN	New model checkpoint	Replace v1
action_scaler.json	COPY	From IQL run dir	Still needed for denormalization
baseline.json	COPY	Airport layout template	Unchanged
des_engine.py	COPY	Simulation core	VALIDATE against Unity
des_output.py	COPY	write_stats, write_csv	Unchanged

Table 1 – continued

File	Source	Purpose	v2 Changes
parse_iql_v2.py	COPY	featurize_absconfig(), ALL_CHECKPOINTS	Add schedule_to_obs()
iata_reward.py	MODIFY	compute_reward_iata(), IATA_LOS	Replace with v2 direct reward
structural_decoder.py	MODIFY	action_to_aeroconfig_struct	Remove int(round()) during training
train_ppo_v2.py	NEW	Training script	Complete rewrite with schedule env

5.2 Node Backend (EXISTING REPO)

File	Purpose	v2 Changes
services/ pythonMlClient.js	HTTP client to Python	Add inferSchedule(schedule) method
routes/diagnose.js	/api/diagnose endpoint	Accept schedule array, not single ABS
services/ llmService.js	Gemini/LLM integration	Update prompt template for schedule context
.env	Environment variables	Add PYTHON_ML_URL, GEMINI_API_KEY

5.3 Unity Simulation (EXISTING — VALIDATION TARGET)

Component	Validation Check
SimClock.cs	Time step matches Python DES delta
PassengerGenerator.cs	Gamma distribution params match Python
CheckpointController.cs	Queue cap enforcement matches Python
StationController.cs	Service time formula matches Python
FlowGraph.cs	Next_Anchor routing matches Python
StatsAggregator.cs	p95 computation matches Python

6. Unity vs Python DES Validation Checklist

CRITICAL: Before training v2, an agentic AI must validate that Python DES produces identical (or statistically equivalent) results to Unity for the same inputs.

6.1 Deterministic Test Protocol

```
# Test 1: Single flight, deterministic seed
aero_config = load_baseline()
abs_config = load_absconfig_001()
seed = 42
pax_count = 100

# Run Python DES
```

```

py_stats = run_sim_in_memory(aero_config, abs_config, seed, pax_count)

# Run Unity DES (via API or manual)
unity_stats = unity_api.run_sim(aero_config, abs_config, seed, pax_count)

# Compare
assert abs(py_stats["per_checkpoint"]["1ST-SEC"]["p95"] -
           unity_stats["per_checkpoint"]["1ST-SEC"]["p95"]) < 1.0 # < 1 minute diff

```

6.2 Validation Checklist

#	Check	Python	Unity	Tolerance	Status
1	Passenger arrival gamma distribution	<code>np.random.gamma(2, 15)</code>	<code>UnityEngine.Random.Gamma(2, 15)</code>	± 0.1 mean	☐
2	Queue discipline (FIFO)	<code>collections.deque</code>	<code>Queue<T></code>	Identical order	☐
3	Max_Queue_Cap enforcement	Drop if <code>len(queue) >= cap</code>	Drop if <code>queue.Count >= cap</code>	Identical drop rate	☐
4	Service time formula	<code>avg_time / efficiency * random(0.8, 1.2)</code>	Same	$\pm 5\%$ mean	☐
5	Efficiency_Factor application	Multiplicative	Multiplicative	Identical	☐
6	Flow routing (Next_Anchor)	Graph traversal	Graph traversal	Identical paths	☐
7	Parallel station selection	Round-robin	Round-robin	Identical	☐
8	p95 computation	<code>np.percentile(times, 95)</code>	Same	± 0.5 min	☐
9	Throughput counting	<code>len(completed)</code>	<code>completed.Count</code>	Identical	☐
10	Completion rate	<code>completed / generated</code>	<code>completed / generated</code>	Identical	☐
11	Multi-flight aggregation	Sum records, then compute stats	Same	Identical	☐
12	Task probability execution	<code>if random() < prob: execute</code>	<code>if Random.value < prob: execute</code>	$\pm 2\%$ task count	☐

6.3 Known Divergence Points

Issue	Python	Unity	Resolution
collapse_to_one structural op	Merges 3 stations into 1	Must merge identically	Validate structural_decoder matches Unity logic
enforce_cap	Limits to max_allowed stations	Must limit identically	Check cap enforcement in Unity
Station naming convention	DEP_1ST_SEC_LANE_01_A	First Lane	CRITICAL: Name mapping must be consistent for comparison
Random seed propagation	seed + flow_idx * 10000	Must match	Document Unity seed formula

7. Training Specification

7.1 Environment Class (v2)

```

class AirportEnv(gymnasium.Env):
    metadata = {"render_modes": []}

    def __init__(self, schedule_sampler, baseline_cfg, action_scaler,
                 pax_range=(50, 500), flight_range=(1, 5)):
        super().__init__()
        self.schedule_sampler = schedule_sampler
        self.baseline_cfg = baseline_cfg
        self.a_min, self.a_max, self.a_range = action_scaler
        self.pax_range = pax_range
        self.flight_range = flight_range

        self.observation_space = spaces.Box(low=0.0, high=1.0, shape=(15,), dtype=np.float32)
        self.action_space = spaces.Box(low=0.0, high=1.0, shape=(130,), dtype=np.float32)

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        self.schedule = self.schedule_sampler.sample(
            pax_range=self.pax_range,
            flight_range=self.flight_range
        )
        return self._schedule_to_obs(self.schedule), {}

    def step(self, action):
        # Decode action (continuous during training)
        action_norm = np.clip(action, 0.0, 1.0)
        action_real = action_norm * self.a_range + self.a_min
        aero_dict = action_to_aeroconfig_structural(
            action_norm=action_norm,
            action_real=action_real,
            baseline_cfg=self.baseline_cfg,
            episode=0,
            continuous=True # NEW: no int rounding during training
        )

```

```

# Run all flights
all_records = []
for flight in self.schedule:
    records = self._run_flight(aero_dict, flight)
    all_records.extend(records)

# Compute aggregate stats
stats = self._compute_aggregate_stats(all_records)
raw_reward = compute_iata_reward(stats)
norm_reward = raw_reward / 100.0

# Next episode
next_schedule = self.schedule_sampler.sample()
next_obs = self._schedule_to_obs(next_schedule)

return next_obs, norm_reward, True, False, {
    "raw_reward": raw_reward,
    "compliance_rate": compute_compliance_rate(stats),
    "per_checkpoint": stats.get("per_checkpoint", {}),
    "total_pax": sum(f["pax_count"] for f in self.schedule)
}

```

7.2 Training Script Structure

```

def main():
    # 1. Load data
    baseline_cfg = json.load(open(BASELINE_PATH))
    action_scaler = load_action_scaler(ACTION_SCALER_DIR)

    # 2. Create sampler
    sampler = ScheduleSampler(ABS_CONFIG_DIR)

    # 3. Create env
    def make_env():
        return AirportEnv(
            schedule_sampler=sampler,
            baseline_cfg=baseline_cfg,
            action_scaler=action_scaler,
            pax_range=(50, 500),
            flight_range=(1, 5)
        )
    env = DummyVecEnv([make_env])

    # 4. Create PPO (no warm-start)
    ppo = PPO(
        policy="MlpPolicy",
        env=env,
        learning_rate=get_linear_fn(3e-4, 1e-5, 1.0),
        n_steps=512,
        batch_size=128,
    )

```

```

n_epochs=4,
gamma=0.99,
clip_range=0.1,
ent_coef=0.01,
vf_coef=0.5,
policy_kwargs=dict(net_arch=[256, 256, 256]),
verbose=1,
tensorboard_log="./ppo_airport_v2/"
)

# 5. Train
eval_cb = EvalCallback(
    eval_env_fn=make_env,
    eval_every=500,
    output_dir=OUTPUT_DIR
)
early_stop = EarlyStopCallback(patience=10, min_delta=2.0, eval_every=500)

ppo.learn(total_timesteps=25_000, callback=CallbackList([eval_cb, early_stop]))
ppo.save("ppo_v2_schedule_aware")

```

7.3 Evaluation Callback (v2)

```

class EvalCallback(BaseCallback):
    def _on_step(self):
        if self.n_calls % self.eval_every != 0:
            return True

        # Evaluate on fixed test schedules
        test_schedules = self._load_test_schedules() # 10 diverse scenarios

        compliance_rates = []
        raw_rewards = []

        for schedule in test_schedules:
            obs = self._schedule_to_obs(schedule)
            action, _ = self.model.predict(obs[None, :], deterministic=True)
            aero = self._decode(action)

            stats = self._run_schedule(aero, schedule)
            compliance_rates.append(compute_compliance_rate(stats))
            raw_rewards.append(compute_iata_reward(stats))

        mean_compliance = np.mean(compliance_rates)
        mean_reward = np.mean(raw_rewards)

        print(f"Step {self.n_calls}: Compliance={mean_compliance:.2%}, Reward={mean_reward:.1f}")

        if mean_compliance > self.best_compliance:
            self.best_compliance = mean_compliance
            self.model.save("best_model_v2")

```

```
return True
```

8. Inference & API Specification

8.1 Python Service Endpoints

```
# POST /infer
# Body: { "schedule": [...] }
# Response: { "aero_config": {...}, "action_norm": [...] }

# POST /simulate
# Body: { "aero_config": {...}, "schedule": [...], "pax_count": int }
# Response: { "raw_reward": float, "compliance_rate": float, "per_checkpoint": {...} }

# POST /diagnose
# Body: { "schedule": [...], "current_aero": {...} }
# Response: { "summary": str, "checkpoints": [...], "recommended_aero": {...} }
```

8.2 Node Backend Flow

```
// 1. Operator uploads schedule (or fetches from API)
const schedule = await fetchAirlineAPI(tomorrow);

// 2. Get baseline performance (simulate current config)
const baselineSim = await pythonService.simulate({
  aero_config: currentAero,
  schedule: schedule
});

// 3. Get PPO recommendation
const { aero_config: recommendedAero } = await pythonService.infer({
  schedule: schedule
});

// 4. Simulate recommended config
const recommendedSim = await pythonService.simulate({
  aero_config: recommendedAero,
  schedule: schedule
});

// 5. Gate: only recommend if it improves
let finalAero = null;
let summary = "";

if (recommendedSim.raw_reward > baselineSim.raw_reward) {
  finalAero = recommendedAero;
  summary = `AI improved reward by ${recommendedSim.raw_reward - baselineSim.raw_reward.toFixed(1)}
  ↪ points.`;
}
```



```

} else {
  finalAero = null;
  summary = `Current config is already optimized (${baselineSim.raw_reward.toFixed(1)}). No changes
    ↪ recommended.`;
}

// 6. Build checkpoint diff for LLM
const diagnostics = buildCheckpointDiff(baselineSim, recommendedSim);

// 7. LLM explanation
const explanation = await llmService.generate({
  summary: summary,
  schedule: schedule,
  diagnostics: diagnostics,
  compliance: {
    baseline: baselineSim.compliance_rate,
    recommended: recommendedSim.compliance_rate
  }
});

// 8. Return to dashboard
return {
  summary: explanation,
  checkpoints: diagnostics,
  recommendedAero: finalAero,
  baselinePerformance: baselineSim,
  recommendedPerformance: recommendedSim,
  shouldApply: finalAero !== null
};

```

8.3 LLM Prompt Template (v2)

You are an airport operations advisor. You will receive a flight schedule and performance data for two configurations: BASELINE (current) and RECOMMENDED (AI-generated).

CRITICAL RULES:

- Reward scale: 0 = perfect, -500 = mediocre, -36,000 = broken. HIGHER is BETTER.
- If recommended_reward < baseline_reward, the AI made things WORSE. Say so explicitly.
- Compliance rate = % of checkpoints within IATA wait time bands. Higher is better.
- Only suggest changes that MATCH the AI's actual recommendations. Do not invent generic advice.

SCHEDULE:

{schedule_summary}

BASELINE PERFORMANCE:

- Reward: {baseline_reward}
- Compliance: {baseline_compliance}%
- Worst bottleneck: {baseline_worst_ck} = {baseline_worst_p95} min

RECOMMENDED PERFORMANCE:

- Reward: {recommended_reward}

```
- Compliance: {recommended_compliance}%  
- Worst bottleneck: {recommended_worst_ck} = {recommended_worst_p95} min
```

PER-CHECKPOINT CHANGES:
{checkpoint_diff_table}

Explain in 2-3 sentences what changed and whether it helped or hurt.
If the AI recommendation is worse, tell the operator to keep their current config.

9. Known Issues & Fixes

9.1 v1 Issues (Fixed in v2)

Issue	v1 Symptom	v2 Fix
Log normalization destroys shaping	<code>explained_variance = 0</code>	Remove log normalization
<code>int(round())</code> kills gradients	Policy can't refine staffing	Continuous decoder during training
Fixed PAX (50/100)	Model fails at 500+ PAX	Variable PAX in schedule sampler
Single-flight episodes	No scale awareness	Multi-flight aggregate episodes
7-dim observation	No demand magnitude	15-dim with PAX, flights, peak hour
IQL warm-start	-497 baseline, mediocre	Discarded; train from scratch or linear baseline
LLM hallucination	"Great news!" on worse config	Gate logic + explicit reward scale in prompt

9.2 v2 Risks

Risk	Mitigation
Training takes 30+ min per run	Use cloud GPU (Colab/Kaggle)
Unity DES diverges from Python	Validate checklist (Section 6) before training
Overfitting to training schedules	Evaluate on held-out real API schedules
Action space too large (130-dim)	Monitor <code>clip_fraction</code> ; reduce if >0.5
Value net still dead	Check <code>explained_variance</code> after 1k steps; if still <0.1 , increase <code>vf_coef</code> to 1.0

Appendix: Reward Math

v1 Reward (Deprecated)

```
raw_reward = base_simulation_output # Typically -1000 to -36,000  
shifted = raw_reward + 1000         # Make positive
```

```
log_val = ln(shifted)                # Compress
normalized = (log_val - log_mean) / log_std  # Z-score

Per-checkpoint shaping (+12/-20) is invisible because:
12 / (shifted_range) ≈ 0.001 → after log and z-score ≈ 0.0001
```

v2 Reward (Active)

```
For each checkpoint:
  Green (lo <= p95 <= hi): +50
  Over-designed (p95 < lo): -3 * (lo - p95) / lo
  Under-designed (p95 > hi): -25 * (p95 - hi) / hi

Total = sum(checkpoint_rewards) + 0.05*throughput + 20.0*completion_rate

Normalized for PPO = Total / 100.0 # Scales to [-5, 5] range

Example:
10 checkpoints, 8 green, 2 under-designed by 20%:
Total = 8*50 + 2*(-25*0.2) + 0.05*800 + 20*0.95
      = 400 - 10 + 40 + 19 = 449
Normalized = 4.49 (strong positive signal)
```

Document Control

Version	Date	Author	Changes
1.0	2026-05-27	PPO Training Session	Initial fast-iteration specs
2.0	2026-06-05	System Architecture Review	Schedule-aware redesign, v1→v2 transition

Next Action: Agentic AI to validate Python DES against Unity using Section 6 checklist. Once validated, proceed with v2 training script implementation.